

Color Names in HTML and CSS

1. What are Color Names?

Color names are predefined names used in HTML and CSS to set colors for elements such as text, background, borders, etc., without using color codes (like hex, RGB).

2. Syntax Example

```
<p style="color: red;">This is red text</p>
```

```
<div style="background-color: lightblue;">Light Blue Background</div>
```

3. Commonly Used Color Names

Color Name

Preview

Black

Black

White

White

Red

Red

Green

Green

Blue

Blue

Yellow

Yellow

Cyan

Cyan

Magenta

Magenta

Orange

Orange

Purple

Purple

Pink

Pink

Gray

Gray

Brown

Brown

4. Total Number of Color Names

- HTML and CSS support **147 named colors**, including standard and extended colors.

5. Case Sensitivity

- Color names are **not case-sensitive**.

<p style="color: RED;">Red text</p>

<p style="color: red;">Also red text</p>

6. Where Color Names Are Used

- Text color: color
- Background color: background-color
- Border color: border-color
- Table styling, divs, buttons, etc.

7. Tips

- Easy to remember but **less flexible** than HEX/RGB/HSLA when precision is needed.
- Use for simple designs or quick prototypes.

8. Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Color Names Example</title>
```

```
</head>
```

```
<body style="background-color: lightgray;">
```

```
<h1 style="color: navy;">Welcome to USBM</h1>
<p style="color: darkgreen;">We teach using fun and colors! </p>
</body>
</html>
```

1. Hex Codes (Hexadecimal Colors)

What is a Hex Code?

- A Hex Code defines a color using a **6-digit** combination of numbers and letters.
- Starts with # followed by values for **Red, Green, and Blue**.

Syntax:

color: #RRGGBB;

Example:

color: #FF0000; /* Red */

background-color: #00FF00; /* Green */

border: 2px solid #0000FF; /* Blue */

Breakdown:

- #FF0000 = Red
- #00FF00 = Green
- #0000FF = Blue
- #000000 = Black
- #FFFFFF = White

2. RGB and RGBA

What is RGB?

- Stands for **Red, Green, Blue**

- Values range from **0 to 255** for each color.

Syntax:

color: rgb(red, green, blue);

Example:

color: rgb(255, 0, 0); /* Red */

color: rgb(0, 255, 0); /* Green */

color: rgb(0, 0, 255); /* Blue */

What is RGBA?

- Same as RGB but includes **Alpha (opacity)**
- Alpha ranges from **0 (transparent)** to **1 (opaque)**

Syntax:

color: rgba(red, green, blue, alpha);

Example:

background-color: rgba(255, 0, 0, 0.5); /* Semi-transparent red */

3. HSL and HSLA

What is HSL?

- Stands for **Hue, Saturation, Lightness**

- **Hue: Angle from 0 to 360**

(0 = red, 120 = green, 240 = blue)

- **Saturation: % of color intensity (0% = gray, 100% = full color)**

- **Lightness: % of brightness (0% = black, 100% = white)**

Syntax:

color: hsl(hue, saturation%, lightness%);

Example:

color: hsl (0, 100%, 50%); /* Red */

color: hsl (120, 100%, 50%); /* Green */

color: hsl (240, 100%, 50%); /* Blue */

What is HSLA?

- Adds Alpha (transparency) to HSL.

Syntax:

color: hsla (hue, saturation%, lightness%, alpha);

Example:

background-color: hsla(240, 100%, 50%, 0.5); /* Semi-transparent blue */

Summary Table

Format	Syntax Example	Use Case
Hex	#FF0000	Short, common in web design
RGB	rgb (255, 0, 0)	Clear intensity control
RGBA	rgba (255, 0, 0, 0.5)	Transparency with RGB
HSL	hsl (0, 100%, 50%)	Easier to adjust brightness/saturation
HSLA	hsla (0, 100%, 50%, 0.3)	Transparency with HSL

1. Font-family

Purpose: Specifies the font type used for text.

Syntax:

font-family: "Arial", "Helvetica", sans-serif;

Details:

- You can specify multiple fonts separated by commas (fallbacks).
- If a font name has spaces, wrap it in quotes.
- Always end with a **generic family**:
 - serif, sans-serif, monospace, cursive, fantasy.

Example:

```
p {  
  font-family: "Times New Roman", serif;  
}
```

2. Font-size

Purpose: Sets the size of the font.

Syntax:

font-size: 16px;

Units:

- **Absolute units:** px, pt, cm
- **Relative units:** em, rem, %, vw

Common Values:

- small, medium, large, x-large

Example:

```
h1 {  
  font-size: 2em; /* 2 times the parent size */  
}
```

3. Font-weight

Purpose: Sets the thickness/boldness of the font.

Syntax:

font-weight: bold;

Values:

- Keywords: normal, bold, lighter, bolder
- Numeric: 100 to 900 (in steps of 100)

Example:

```
strong {  
  font-weight: 700;  
}
```

4. Font-style

Purpose: Specifies the style of the font (italic, oblique, or normal).

Syntax:

font-style: italic;

Values:

- normal
- italic – makes text slanted
- oblique – also slanted but slightly different rendering

Example:

```
em {  
  font-style: italic;  
}
```

5. Text-align

Purpose: Aligns text horizontally.

Syntax:

text-align: center;

Values:

- left (default for LTR languages)
- right
- center
- justify – spreads text evenly across the line

Example:

```
div {  
  text-align: justify;  
}
```

6. Text-decoration

Purpose: Adds or removes decorations (like underline) from text.

Syntax:

text-decoration: underline;

Values:

- none – removes all decorations
- underline
- overline
- line-through

Example:

```
a {  
  text-decoration: none;  
}
```

7. Text-transform

Purpose: Controls capitalization of text.

Syntax:

text-transform: uppercase;

Values:

- none – default
- uppercase – all letters capital
- lowercase – all letters small
- capitalize – first letter of each word capital

Example:

```
h2 {  
  text-transform: capitalize;  
}
```

8. Line-height

Purpose: Sets the height of a line of text (line spacing).

Syntax:

line-height: 1.5;

Units:

- Unitless (recommended) – relative to font-size
- With units: px, em, etc.

Example:

```
p {  
  line-height: 1.6; /* 160% of font-size */  
}
```

9. Letter-spacing

Purpose: Sets spacing between characters.

Syntax:

letter-spacing: 2px;

Values:

- Positive value increases space
- Negative value decreases space

Example:

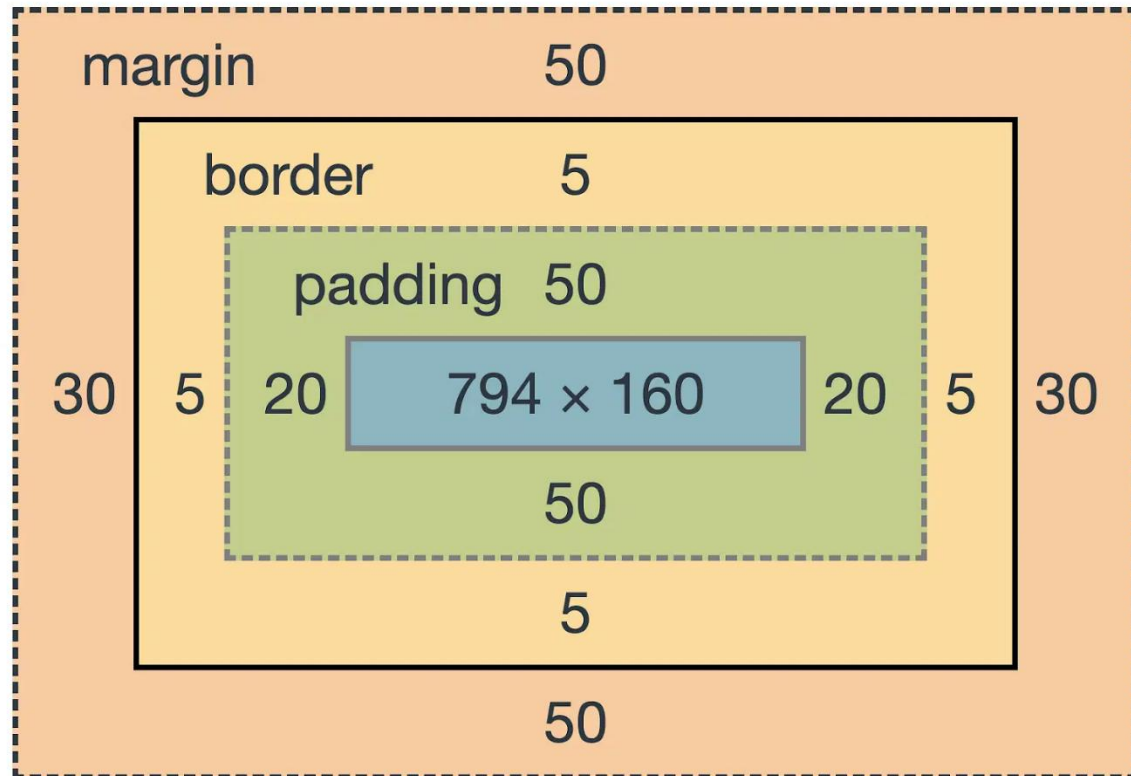
```
h3 {  
  letter-spacing: 1px;  
}
```

Combined Example:

```
body {  
    font-family: "Roboto", sans-serif;  
    font-size: 16px;  
    font-weight: 400;  
    font-style: normal;  
    text-align: left;  
    text-decoration: none;  
    text-transform: none;  
    line-height: 1.5;  
    letter-spacing: 0.5px;  
}
```

What is the CSS Box Model?

The **CSS Box Model** is the foundation of layout in web design. Every HTML element is considered a box consisting of:



1. Margin

Purpose: Creates space **outside** the element's border.

Syntax:

margin: 10px;

margin: top right bottom left;

margin: 10px 20px; /* top-bottom 10px, left-right 20px */

margin: 0 auto; /* center block horizontally */

2. Padding

Purpose: Creates space **inside** the element, between content and border.

Syntax:

padding: 15px;

padding: 10px 15px 5px 0;

3. Border

Purpose: Creates a visible boundary around padding and content.

Syntax:

border: 2px solid red;

Individual properties:

- border-width
- border-style (solid, dashed, dotted, none, double)
- border-color

border-top: 1px dashed green;

4. Box-sizing

Controls how total width and height are calculated.

a. content-box (default)

- width/height apply to content only.
- Padding and border are **added** outside.

b. border-box

- width/height include content + padding + border.
- Easier for layout control.

Example:

```
.box1 {  
  width: 200px;  
  padding: 20px;  
  border: 10px solid blue;  
  box-sizing: content-box; /* Total = 200 + 20*2 + 10*2 = 260px */  
}
```

```
.box2 {  
  width: 200px;  
  padding: 20px;  
  border: 10px solid red;  
  box-sizing: border-box; /* Total = 200px */  
}
```

5. Width and Height

Set the size of elements:

```
div {  
  width: 300px;  
  height: 150px;  
}
```

Units: px, %, em, rem, vw, vh

6. Min-width / Max-width and Min-height / Max-height

min-width, min-height

- Ensures element never shrinks below this size.

max-width, max-height

- Restricts the element from growing beyond this size.

Example:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

7. Outline vs Border

Feature	Border	Outline
Inside box?	Inside box model	Outside element box
Takes space?	Yes (affects layout)	No (doesn't take space)
Style	All border styles	Limited (solid, dashed...)
Individual Sides?	Yes	No
Use case	Layout and design	Focus indicators, highlights

Example:

```
input: focus {  
  outline: 2px solid blue;  
}
```

Complete Example:

```
<!DOCTYPE html>  
  
<html>
```

```
<head>
<style>
  . box {
    width: 200px;
    height: 100px;
    margin: 20px;
    padding: 15px;
    border: 5px solid black;
    outline: 3px dashed red;
    box-sizing: border-box;
    background-color: lightyellow;
  }
</style>
</head>
<body>
  <div class="box">Box Model Example</div>
</body>
</html>
```


CSS Units

CSS units define the measurement used for properties like width, height, margin, padding, font-size, etc. Units are categorized into **Absolute** and **Relative** units.

1. Absolute Units

These units are fixed and do not scale with screen size or user settings.

Unit	Description	Example
px	Pixels (1px = 1 dot on screen)	font-size: 16px;
pt	Points (1pt = 1/72 inch)	font-size: 12pt;
cm	Centimeters	width: 5cm;
mm	Millimeters	margin: 10mm;
in	Inches (1in = 2.54cm)	width: 1in;
pc	Picas (1pc = 12pt)	font-size: 1pc;

Use **absolute units** when printing or when precise physical dimensions are needed.

2. Relative Units

These are based on the size of something else — usually the parent or root element. More flexible and responsive.

Unit	Description	Example
%	Percentage of the parent element	width: 50%;
em	Relative to the font-size of the parent element	padding: 2em;
rem	Relative to the root element's font-size	font-size: 1.5rem;
vw	1% of the viewport width	width: 80vw;
vh	1% of the viewport height	height: 50vh;
vmin	1% of the smaller dimension of viewport	font-size: 5vmin;
vmax	1% of the larger dimension of viewport	font-size: 5vmax;
ch	Width of the character 0 in the current font	width: 40ch;
ex	Height of the character x in the current font	line-height: 2ex;

Use **relative units** for responsive design and better accessibility.

em vs rem – Key Difference

Feature	em	rem
Relative to	Parent element's font size	Root element (html) font size
Cascading effect	Yes (can compound)	No (remains constant)
Example	font-size: 2em; = 2× parent size	font-size: 2rem; = 2× root size

Viewport Units – Use Cases

Unit	Common Use Case
vw	Full-width layouts, responsive banners
vh	Full-screen sections
vmin/vmax	Scalable font sizes or containers

Examples

```
body {  
  font-size: 16px; /* base */  
}
```

```
.container {  
  width: 80vw;    /* 80% of viewport width */  
  height: 50vh;  /* 50% of viewport height */  
  padding: 2em;   /* 2 times parent's font-size */  
  font-size: 1.2rem; /* 1.2 times root font-size */  
  border: 1px solid black;  
}
```

Practices

- Use rem for font sizes to maintain consistent scaling.
- Use % for layout width to adapt to parent containers.
- Use vw, vh for full-screen responsiveness.
- Avoid mixing too many units in one file unless necessary.